

EXPERIMENT NO. 6

Sort a List and Find the Maximum Element

Programming Language: C

Name : Bodham susma

Register No. : 192571065

Aim

To write a C program that sorts a list using an efficient sorting algorithm (Quicksort through qsort) and then finds the maximum element of the sorted list.

Question

You have an algorithm that process a list of numbers. It firsts sorts the list using an efficient sorting algorithm and then finds the maximum element in sorted list. Write the code for the same.

Test Cases

Empty List

Input: []

Expected Output: None or an appropriate message indicating that the list is empty.

Single Element List

Input: [5]

Expected Output: 5

All Elements are the Same

Input: [3, 3, 3, 3, 3]

Expected Output: 3

Algorithm

Step 1: Start.

Step 2: Pass the array nums and its size to the function findMaxUsingSort.

Step 3: If size <= 0, print "The list is empty." and return.

Step 4: Sort the array using qsort with the comparison function compare.

Step 5: In the sorted array the maximum element is at the last index: maxElement = nums[size - 1].

Step 6: Print maxElement.

Step 7: Stop.

Program (C)

```
#include <stdio.h>
#include <stdlib.h>

// Comparison function for qsort
int compare(const void* a, const void* b) {
    return (*(int*)a - *(int*)b);
}

// Function to sort the array and find the maximum element
void findMaxUsingSort(int* nums, int size) {
    if (size <= 0) {
        printf("The list is empty.\n");
        return;
    }

    // Sort the list using Quicksort
    qsort(nums, size, sizeof(int), compare);

    // In a sorted list, the maximum element is at the last index
    int maxElement = nums[size - 1];
    printf("Expected Output: %d\n", maxElement);
}

int main() {
    // Test Case 1: Empty List
    int case1[] = {};
    printf("Test Case 1 -> ");
    findMaxUsingSort(case1, 0);

    // Test Case 2: Single Element List
    int case2[] = {5};
    printf("Test Case 2 -> ");
    findMaxUsingSort(case2, 1);

    // Test Case 3: All Elements are the Same
    int case3[] = {3, 3, 3, 3, 3};
    printf("Test Case 3 -> ");
    findMaxUsingSort(case3, 5);

    return 0;
}
```

Output

```
Test Case 1 -> The list is empty.
Test Case 2 -> Expected Output: 5
Test Case 3 -> Expected Output: 3

=== Code Execution Successful ===
```

Complexity Analysis

Time Complexity: $O(n \log n)$

Space Complexity: $O(\log n)$

Result

Thus the program was executed successfully and the output was verified.